

ECEN 4/5632 — Problem Set 4 Solutions

FIR Design & Windowing (Lecture 7) + IIR Design & Bilinear Transform (Lecture 8)

Instructor solutions. Numerical answers were checked in Python (NumPy/SciPy); expect small floating-point differences in student work.

Part A — FIR Filters, Linear Phase, and Windowing

A1. Linear phase and group delay

(a) Split the sum using the symmetry $h[n] = h[N - 1 - n]$, pairing terms n and $N - 1 - n$ about the center $c = (N - 1)/2$:

$$H(e^{j\omega}) = \sum_{n=0}^{N-1} h[n] e^{-j\omega n} = e^{-j\omega c} \sum_{n=0}^{N-1} h[n] e^{-j\omega(n-c)}$$

Because $h[n] = h[N - 1 - n]$, the bracketed sum is symmetric about $n = c$, so pairing $n = c + k$ with $n = c - k$ gives terms $h[c + k] (e^{-j\omega k} + e^{j\omega k}) = 2h[c + k] \cos(\omega k)$, which is real. Hence

$$H(e^{j\omega}) = e^{-j\omega(N-1)/2} A(\omega), \quad A(\omega) \in \mathbb{R}$$

(b) $\angle H(e^{j\omega}) = -\omega(N - 1)/2 + \angle A(\omega)$. Since $A(\omega)$ is real, $\angle A(\omega)$ is either 0 or π (jumping by π only where $A(\omega)$ changes sign). Away from those sign changes,

$$\tau_g(\omega) = -\frac{d}{d\omega} \left[-\omega \frac{N - 1}{2} \right] = \frac{N - 1}{2} = \text{constant}$$

(c) $\tau_g = (63 - 1)/2 = 31$ samples. At $f_s = 8$ kHz, $T = 1/8000$ s, so delay = $31/8000 = 3.875$ ms.

(d) Linear phase means every passband frequency is delayed by the *same* number of samples — not zero samples. A concrete example: a linear-phase FIR used to condition a radar or lidar return before peak-detection for range estimation. The absolute range estimate must be corrected by subtracting the known constant delay τ_g samples (equivalently $\tau_g \cdot T$ seconds of two-way travel time); failing to do so biases every range estimate by the same constant offset.

A2. Ideal lowpass impulse response

**** (a) ****

$$h_d[n] = \frac{1}{2\pi} \int_{-\omega_c}^{\omega_c} e^{j\omega n} d\omega = \frac{1}{2\pi} \left[\frac{e^{j\omega n}}{jn} \right]_{-\omega_c}^{\omega_c} = \frac{e^{j\omega_c n} - e^{-j\omega_c n}}{2\pi j n} = \frac{\sin(\omega_c n)}{\pi n}, \quad n \neq 0$$

For $n = 0$, integrate directly: $h_d[0] = \frac{1}{2\pi} \int_{-\omega_c}^{\omega_c} d\omega = \frac{\omega_c}{\pi}$, which also equals

$\lim_{n \rightarrow 0} \sin(\omega_c n)/(\pi n)$ by L'Hôpital or the standard sinc limit.

(b) $h_d[-n] = \sin(-\omega_c n)/(-\pi n) = \sin(\omega_c n)/(\pi n) = h_d[n]$, so $h_d[n]$ is even. This symmetry about $n = 0$ is required for zero phase: $H_d(e^{j\omega})$ is purely real (no linear-phase term), which by the conjugate-symmetry/evenness duality forces $h_d[n]$ real and even.

(c) $\omega_c = \pi/2 \Rightarrow h_d[0] = 0.5$. Also $h_d[\pm 1] = \sin(\pi/2)/\pi = 1/\pi \approx 0.3183$. Energy check: $h_d[0]^2 = 0.25$, $h_d[1]^2 = h_d[-1]^2 \approx 0.1013$ each. Of the "energy" contained in just these three samples, the center sample holds $0.25/(0.25 + 2 \times 0.1013) \approx 55\%$ — illustrating that even for the ideal filter, a large fraction of the impulse response energy is concentrated near $n = 0$, though (as the lecture emphasizes) the *tails* never vanish, which is exactly the problem truncation must solve.

A3. Truncation, shifting, Gibbs phenomenon

(a) Centering at $M/2$ and shifting for causality:

$$h_d[n] = \frac{\sin(\omega_c(n - M/2))}{\pi(n - M/2)}, \quad n = 0, \dots, M$$

Group delay $\tau_g = M/2 = (N - 1)/2$ samples (with $N = M + 1$ taps), matching A1(c).

(b)-(c) Representative Python:

```
python

import numpy as np
from scipy.signal import freqz

def rect_fir(N, wc):
    M = N - 1
    n = np.arange(N)
    h = np.sinc(wc*(n - M/2)/np.pi) * (wc/np.pi)  # np.sinc(x) = sin(pi x)/(pi x)
    return h

for N in [21, 51, 101, 201]:
    h = rect_fir(N, 0.3*np.pi)
    w, H = freqz(h, worN=4096)
    print(N, 20*np.log10(np.max(np.abs(H))))
```

Running this shows the **peak passband value stays pinned near +0.7 to +0.9 dB overshoot** for all four lengths (the exact number depends on how finely you sample ω near the discontinuity), while the width of the *ripple region* shrinks roughly as $1/N$. This is the numerical face of the classical Gibbs result: for a jump discontinuity of height Δ , the overshoot asymptotically approaches about **8.9%** of Δ (the "Gibbs constant," $\approx 1.0895 \times$ the ideal jump), independent of N . In dB, an amplitude overshoot to ≈ 1.09 corresponds to $20 \log_{10}(1.09) \approx 0.75$ dB — consistent with the plot in Lecture 7 (slide 12) showing all three

curves overshoot to about the same peak near the cutoff.

(d) The claim is false: increasing N narrows the transition and compresses the ripple closer to ω_c , but the **peak** overshoot (both in the passband and the first stopband sidelobe) converges to a fixed value (~ 13 dB down for the first stopband sidelobe of a rectangular window, $\sim 9\%$ amplitude overshoot near jumps) and does not shrink further. To reduce peak ripple you must change the *window*, not just the length.

A4. Window comparison and length budgeting

(a) Required stopband attenuation is 50 dB. The first-sidelobe column tells us the **best-case, unimprovable-by-length floor** for each window:

- Rectangular (-13 dB) and Bartlett (-26 dB): **cannot** meet 50 dB regardless of N (their first sidelobe alone is already above -50 dB).
- Hann (-31 dB): also cannot meet 50 dB.
- Hamming (-43 dB): still short of 50 dB (close, but the spec calls for *at least* 50 dB, so Hamming is a marginal/insufficient choice).
- Blackman (-58 dB): comfortably exceeds 50 dB and is the safe choice.

(Instructor note: some students may reasonably argue Hamming is "close enough" pending a numerical check with a real design tool such as `scipy.signal.firwin` with a Kaiser-equivalent or explicit window; the expected reasoning is that Blackman is the only window in the table whose intrinsic sidelobe floor is safely below the -50 dB target.)

(b) Using Blackman, main-lobe width $\approx 12\pi/N$. Setting this equal to (twice) the transition width is a common convention; using the direct convention $\Delta\omega \approx 12\pi/N$ with $\Delta\omega = 0.1\pi$:

$$N \approx \frac{12\pi}{0.1\pi} = 120$$

So $N \approx 120$ -121 taps (round up, and to an odd number for a Type-I linear-phase filter if a single center tap is desired).

(c) Group delay $= (N - 1)/2 \approx 60$ samples for $N = 121$.

(d) Representative check:

```
python
```

```

from scipy.signal import firwin, freqz
import numpy as np

N = 121
wc = (0.35 + 0.45)/2 * np.pi # cutoff at the transition-band center
h = firwin(N, wc/np.pi, window='blackman')
w, H = freqz(h, worN=8192)
# Verify: |H| in dB at w = 0.45*pi should be <= -50 dB
idx = np.argmin(np.abs(w - 0.45*np.pi))
print(20*np.log10(np.abs(H[idx])))

```

This should print a value at or below about -55 to -58 dB, confirming the 50 dB spec is met with margin.

A5. Window method as periodic convolution

(a) $W(e^{j\omega})$ acts like a smoothing kernel in frequency. A narrow W main lobe means each point of H_w is a weighted average of H_d over a *small* neighborhood of frequencies, so the sharp edge of H_d at ω_c is only slightly "blurred" — the transition stays narrow. A wide main lobe averages over a larger neighborhood, spreading the edge over a wider band.

(b) The sidelobes of W are what "leak" energy from the passband into the stopband (and vice versa) during the convolution — a stopband frequency picks up a small but nonzero contribution from the (much larger) passband value of H_d weighted by W 's sidelobe there. Lower sidelobes mean less leakage, hence less ripple. The tension arises because a window's time-domain taper that goes smoothly to zero at its edges (low sidelobes, e.g. Blackman) is, for fixed length N , "less rectangular," which necessarily broadens its main lobe — you cannot simultaneously make the taper both very sudden (narrow main lobe) and very smooth (low sidelobes) within the same finite support.

A6. Conv1d and the sliding dot product

(a) Compare term-by-term: $y_{\text{DSP}}[n] = \sum_{k=0}^M h[k]x[n-k]$ vs. $y_{\text{torch}}[n] = \sum_k w[k]x[n+k]$. Substituting $k' = M - k$ in the DSP sum: $y_{\text{DSP}}[n] = \sum_{k'=0}^M h[M-k']x[n-M+k']$. Letting $m = n - M$ (i.e., evaluating at a shifted index) and matching to the torch form gives $w[k] = h[M-k]$ — the taps must be **time-reversed**. To make the (non-causal-by-default) `conv1d` cross-correlation reproduce a *causal* filter, left-pad the input by $M = N - 1$ samples (so the sliding window only ever "looks back," never forward, relative to the original time axis).

(b)

```
python
```

```

import numpy as np, torch, torch.nn.functional as F
from scipy.signal import firwin

h = firwin(51, 0.3, window='hamming')          # length-51 causal FIR
x = np.random.randn(500).astype(np.float32)

y_np = np.convolve(x, h)[:len(x)]              # causal, length-matched

h_t = torch.tensor(h, dtype=torch.float32)
x_t = torch.tensor(x).view(1,1,-1)
kernel = h_t.flip(0).view(1,1,-1)
x_pad = F.pad(x_t, (len(h)-1, 0))
y_torch = F.conv1d(x_pad, kernel).view(-1).numpy()

print(np.max(np.abs(y_np - y_torch)))          # ~1e-6 - 1e-7

```

(c) Rebuttal: it is not a bug, just a different (and equally valid) naming convention. Deep-learning frameworks implement cross-correlation because in almost all ML use cases the kernel is *learned from data*, not designed analytically — flipping the kernel or not has zero effect on what the network can represent, since gradient descent will simply learn whichever orientation minimizes the loss. The convention only matters when you deliberately want to *drop in* a classically-designed filter's coefficients and reproduce its exact causal behavior, in which case you flip the kernel yourself (part a).

A7. Tensor shapes

(a) 64 clips $\times$ 2 channels $\times$ (3 s $\times$ 44100 Hz) samples: $x \in \mathbb{R}^{64 \times 2 \times 132300}$, i.e., shape $(64, 2, 132300)$.

(b) To apply each of 16 kernels independently to each of the 2 input channels (16 kernels $\times$ 2 channels = 32 independent filtering operations, one kernel per input channel, no mixing across channels), use **depthwise-style grouped convolution**: set $(\text{in_channels}=2, \text{out_channels}=32, \text{groups}=2)$. Weight shape is $(\text{out_channels}, \text{in_channels}/\text{groups}, \text{kernel_size}) = (32, 1, 31)$. Using $(\text{groups}=2)$ ensures each output channel only sees one input channel (so the 16 kernels applied to channel 0 don't mix with channel 1), which is the "independent filter bank per channel" behavior requested. (Using $(\text{groups}=1)$ would instead mix both input channels into every output channel — not what's asked.)

A8. Minute-paper synthesis (sample answer)

*A neural 1-D convolution performs exactly the same local, sliding weighted sum as a designed FIR filter — the arithmetic $y[n] = \sum_k w[k]x[n \pm k]$ is identical. What changes is the source of the weights: in classical FIR design the taps $h[k]$ are derived analytically to satisfy magnitude/phase specifications, whereas a Conv1d layer's weights are free parameters optimized by backpropagation to reduce a task loss. This also typically means giving up

guarantees like exact linear phase or a specified stopband attenuation, in exchange for the ability to adapt the "filter" to whatever the data and task actually require, and to stack many such kernels/channels into deeper, nonlinear feature extractors.*

Part B — IIR Design and the Bilinear Transform

B1. Choosing an analog prototype

(a) **Butterworth** — smooth, monotonic passband (maximally flat), no ripple; a gentle transition is explicitly acceptable, so there's no reason to "spend" ripple anywhere.

(b) **Elliptic** — ripple allowed in *both* bands and steepest transition per order is exactly the elliptic filter's defining advantage (minimum order for a given spec).

(c) **Chebyshev II** — monotonic (ripple-free) passband, equiripple stopband; matches "flat passband, some stopband ripple OK."

(d) **Chebyshev I** — equiripple passband, monotonic stopband; a small controlled passband ripple in exchange for a sharper transition than Butterworth is exactly its design point.

B2. Butterworth order

(a) At $\Omega = \Omega_c$: $|H_c(j\Omega_c)|^2 = 1/(1 + 1) = 1/2 \Rightarrow |H_c(j\Omega_c)| = 1/\sqrt{2}$. In dB: $20 \log_{10}(1/\sqrt{2}) = -10 \log_{10}(2) \approx -3.01$ dB, independent of N .

(b) Require

$$\frac{1}{1 + (\Omega_p/\Omega_c)^{2N}} \geq 10^{-A_p/10} \Rightarrow (\Omega_p/\Omega_c)^{2N} \leq 10^{A_p/10} - 1$$

$$\frac{1}{1 + (\Omega_s/\Omega_c)^{2N}} \leq 10^{-A_s/10} \Rightarrow (\Omega_s/\Omega_c)^{2N} \geq 10^{A_s/10} - 1$$

Dividing the second inequality by the first (both sides positive):

$$\left(\frac{\Omega_s}{\Omega_p}\right)^{2N} \geq \frac{10^{A_s/10} - 1}{10^{A_p/10} - 1}$$

Taking $\log_{10}$ of both sides and solving for N gives the stated formula. N must be rounded **up**, since rounding down would violate at least one of the two inequalities (order can only be increased, never fractional, to guarantee both specs are met).

(c) $A_p = 1, \Omega_p = 1, A_s = 40, \Omega_s = 3$:

$$10^{0.1} - 1 = 0.2589, \quad 10^4 - 1 = 9999$$

$$N \geq \frac{\log_{10}(9999/0.2589)}{2 \log_{10}(3)} = \frac{\log_{10}(38,623)}{2(0.4771)} = \frac{4.587}{0.9542} \approx 4.81$$

Round up: $N = 5$.

B3. LHP maps to the unit disk

(a) With $s = \sigma + j\Omega$:

$$|1 \pm sT/2|^2 = \left(1 \pm \frac{\sigma T}{2}\right)^2 + \left(\frac{\Omega T}{2}\right)^2$$

$$|1 + sT/2|^2 - |1 - sT/2|^2 = \left(1 + \frac{\sigma T}{2}\right)^2 - \left(1 - \frac{\sigma T}{2}\right)^2 = 4 \cdot \frac{\sigma T}{2} = 2\sigma T$$

(using $a^2 - b^2 = (a + b)(a - b)$ with $a = 1$, $b = \sigma T/2$, or directly expanding). Hence:

- $\sigma < 0 \Rightarrow |1 + sT/2|^2 < |1 - sT/2|^2 \Rightarrow |z|^2 = |1 + sT/2|^2 / |1 - sT/2|^2 < 1 \Rightarrow |z| < 1$.
- $\sigma = 0$: the two magnitudes are equal $\Rightarrow |z| = 1$.
- $\sigma > 0 \Rightarrow |z| > 1$.

(b) This means every stable analog pole ($\text{Re}\{s\} < 0$) maps to a stable digital pole ($|z| < 1$), and the imaginary axis (the boundary of stability) maps exactly onto the unit circle (the boundary of digital stability). So any stable analog prototype (Butterworth, Chebyshev, elliptic, ...) is guaranteed, algebraically, to produce a stable digital filter after the substitution — no case-by-case stability check of individual poles is needed.

(c) Impulse invariance ($z = e^{sT}$) also preserves stability, but it maps the *entire* horizontal strips $\text{Im}\{s\} \in (2\pi k/T - \pi/T, 2\pi k/T + \pi/T)$ onto the *same* unit circle for every integer k — i.e., it is many-to-one and therefore **aliases** high analog frequencies into the digital band. The bilinear transform instead maps the entire infinite $j\Omega$ axis one-to-one onto the unit circle exactly once, avoiding aliasing entirely — but at the cost of compressing (**warping**) the infinite analog frequency axis into the finite digital range $(-\pi, \pi)$, which distorts frequency spacing (especially near Nyquist).

B4. Frequency warping relation

(a) Substitute:

$$j\Omega = \frac{2}{T} \cdot \frac{1 - e^{-j\omega}}{1 + e^{-j\omega}}$$

Multiply numerator and denominator by $e^{j\omega/2}$:

$$\frac{1 - e^{-j\omega}}{1 + e^{-j\omega}} = \frac{e^{j\omega/2} - e^{-j\omega/2}}{e^{j\omega/2} + e^{-j\omega/2}} = \frac{2j \sin(\omega/2)}{2 \cos(\omega/2)} = j \tan(\omega/2)$$

so $j\Omega = \frac{2}{T} j \tan(\omega/2) \Rightarrow \Omega = \frac{2}{T} \tan(\omega/2)$. Inverting gives $\omega = 2 \tan^{-1}(\Omega T/2)$.

(b) For small ω , $\tan(\omega/2) \approx \omega/2$, so $\Omega \approx \frac{2}{T} \cdot \frac{\omega}{2} = \omega/T$ — the familiar (unwarped) relation between continuous and discrete frequency.

(c) As $\omega \rightarrow \pi^-$, $\tan(\omega/2) \rightarrow \infty$, so $\Omega \rightarrow \infty$. This means the *entire* remaining analog frequency axis (all frequencies from some large Ω out to infinity) gets compressed into a small

range of ω near π . Practically: if a designer specifies the analog prototype's cutoff using the *numerical* value of the desired digital cutoff (i.e., forgets to prewarp), the achieved digital cutoff will land at a **lower** frequency than intended, and the error grows rapidly as the target digital frequency approaches π (Nyquist).

B5. Prewarping design exercise

(a)

$$\Omega_p = 2 \tan(0.15\pi) = 2 \tan(27^\circ) \approx 2(0.5095) = 1.019 \text{ rad/s}$$

$$\Omega_s = 2 \tan(0.30\pi) = 2 \tan(54^\circ) \approx 2(1.3764) = 2.753 \text{ rad/s}$$

(b) Using the B2(b) formula with $A_p = 1 \text{ dB}$, $A_s = 30 \text{ dB}$:

$$10^{0.1} - 1 = 0.2589, \quad 10^3 - 1 = 999$$

$$N \geq \frac{\log_{10}(999/0.2589)}{2 \log_{10}(2.753/1.019)} = \frac{\log_{10}(3859)}{2 \log_{10}(2.701)} = \frac{3.587}{0.8635} \approx 4.15 \Rightarrow N = 5$$

(c)

```
python

import numpy as np
from scipy.signal import butter, bilinear, freqz

T = 1.0
wp, ws = 0.3*np.pi, 0.6*np.pi
Wp = (2/T)*np.tan(wp/2)      # prewarped edges
Ws = (2/T)*np.tan(ws/2)
N = 5                        # from part (b)

# Design analog prototype with cutoff set by the passband edge (exact -1 dB there)
Omega_c = Wp / (10**(1/10) - 1)**(1/(2*N))
b_a, a_a = butter(N, Omega_c, analog=True)
b_z, a_z = bilinear(b_a, a_a, fs=1/T)

w, H = freqz(b_z, a_z, worN=8192)
idx_p = np.argmin(np.abs(w - wp))
idx_s = np.argmin(np.abs(w - ws))
print("At wp:", 20*np.log10(np.abs(H[idx_p])), "dB")    # should be ~ -1 dB
print("At ws:", 20*np.log10(np.abs(H[idx_s])), "dB")    # should be <= -30 dB (with
```

With $N = 5$ and $\Omega_c \approx 1.167 \text{ rad/s}$, the passband edge lands at essentially -1 dB by construction, and the stopband edge typically comes out attenuated by **more** than 30 dB (roughly 37 dB), because N was rounded up from 4.15 to the next integer.

(d) Repeating **without** prewarping (using $\Omega_p = \omega_p = 0.3\pi \approx 0.9425$, $\Omega_s = \omega_s = 0.6\pi \approx 1.885$ directly as if they were analog rad/s) and re-running the same bilinear pipeline will place the actual -1 dB and -30 dB **digital** points at frequencies **below** the intended 0.3π and 0.6π . Numerically, using $\Omega_c = 0.9425/(0.2589)^{1/10} \approx 1.079$, the achieved -1 dB digital frequency is

$$\omega_{\text{achieved}} = 2 \tan^{-1}(\Omega_c T/2) = 2 \tan^{-1}(0.540) \approx 0.990 \text{ rad} \approx 0.315\pi$$

— only a modest error near $\omega_p = 0.3\pi$ in this example, but the mismatch grows sharply for edges specified closer to $\omega = \pi$ (see B4(c)); students should verify numerically with `freqz` and report the exact discrepancy for their own random seed / chosen edges.

B6. Group delay of the IIR filter

(a)

```
python
from scipy.signal import group_delay
w, gd = group_delay((b_z, a_z))
```

Plotting `gd` vs. `w` shows a **non-flat** curve: delay is largest in the transition band (a "hump," as in Lecture 8 slide 22) and decreases toward the edges — qualitatively different from the FIR case.

(b) The FIR filter's flat group delay means every passband frequency is shifted by the same number of samples, preserving relative waveform shape (peak positions, transient timing). The IIR filter's frequency-dependent delay means different frequency components of a transient arrive at different times, so the *shape* of transients is smeared even though the magnitude response can be nearly identical to an FIR design. For a downstream ML task that only cares about long-term spectral energy (e.g., overall band-power features, bag-of-frequency-features classifiers), this smearing is largely irrelevant. For a task that cares about precise onset timing or transient peak alignment (e.g., R-wave detection timing in ECG, or event localization from a sensor burst), the frequency-dependent group delay can meaningfully corrupt the feature of interest.

B7. Forward-backward filtering

(a) For a real-coefficient filter, $H(e^{-j\omega}) = \overline{H(e^{j\omega})}$ (complex conjugate). The forward-backward response is

$$H_{fb}(e^{j\omega}) = H(e^{j\omega}) H(e^{-j\omega}) = H(e^{j\omega}) \overline{H(e^{j\omega})} = |H(e^{j\omega})|^2$$

which is real and non-negative for all ω , i.e., it has exactly zero phase (no phase to unwrap at all — the imaginary part is identically zero).

(b) If the single-pass filter has $|H(e^{j\omega_c})| = 10^{-3/20} = 0.7079$ (i.e. -3 dB in amplitude), then

$|H_{fb}(e^{j\omega_c})| = |H(e^{j\omega_c})|^2 = 0.501$, whose dB value is $20 \log_{10}(0.501) \approx -6.02$ dB. In general, squaring the amplitude **doubles the dB attenuation**. So if you want the forward-backward filter's final edge to be at -3 dB, design the single-pass filter's edge to be at -1.5 dB (half the target dB value); doubling on the second pass then lands you at the desired -3 dB point.

(c) Reasons `filtfilt` cannot be used in real-time streaming: (1) the backward pass requires the *entire* signal (including future samples relative to any given sample) before it can run, which is impossible in a causal, low-latency streaming system; (2) it must process a finite, already-complete record, so it cannot emit an output sample until the full buffer (or the full relevant chunk) is available, introducing unbounded/unacceptable latency for continuous real-time operation. Reason it is attractive offline: for one-time preprocessing of a whole recorded/stored dataset before training, the zero-phase property preserves exact temporal alignment of transients and features relative to the *unfiltered* reference signal, avoiding the group-delay distortion analyzed in B6.

(d) Representative code:

```
python

import numpy as np
from scipy.signal import lfilter, filtfilt

fs = 1000
t = np.arange(0, 1, 1/fs)
clean = np.exp(-((t-0.5)**2)/(2*0.01**2))      # Gaussian transient
noisy = clean + 0.15*np.sin(2*np.pi*120*t) + 0.1*np.random.randn(len(t))

y_causal = lfilter(b_z, a_z, noisy)
y_fb      = filtfilt(b_z, a_z, noisy)
```

Plotting `clean`, `y_causal`, and `y_fb` together shows the causal output's transient peak shifted later in time by roughly the filter's group delay near that frequency, while the forward-backward output's peak aligns almost exactly with the clean reference's peak (matching Lecture 8, slide 24).

B8. Train/serve skew (sample answer)

(a) Three concrete differences:

1. **Timing/shape of transients:** `filtfilt` output has zero-phase (transients aligned with ground truth), while the causal deployment filter delays and reshapes transients according to its frequency-dependent group delay — features the model learned to associate with "fault at time t " may appear shifted or smeared at inference time.
2. **Effective attenuation:** `filtfilt` applies $|H|^2$, roughly double the dB attenuation of a single causal pass with the same coefficients — so noise/interference levels the model

was trained on are systematically higher at deployment than in training.

3. **Edge effects:** `filtfilt` on finite offline records handles start/end transients via padding/reflection, which single-pass causal streaming does not reproduce, especially for the first $\approx \tau_g$ samples after a stream starts or resets.

(b) Design change: replace the offline `filtfilt` preprocessing with a **causal** IIR (or linear-phase FIR) filter during training as well, so the training distribution exactly matches what the deployed causal filter will produce. Trade-off: this sacrifices the zero-phase alignment advantage during training (transients used for labeling/feature engineering will now be delayed and possibly shape-distorted, exactly as at deployment), which may require re-validating that labels/features are still well time-aligned, or accepting a linear-phase FIR (constant but nonzero delay) as a compromise that keeps waveform *shape* undistorted while still being deployable causally.

B9. FIR vs. IIR vs. learned front end (sample answer, ECG denoising)

Design choice	Strength	Cost/risk	Fit for ECG denoising
Linear-phase FIR	Constant delay preserves QRS-complex shape and timing	Needs more taps for a sharp transition $\rightarrow$ higher latency/compute	Good fit: R-wave timing is diagnostically important, so shape preservation matters more than minimal order
IIR (causal)	Very efficient (low order) sharp notch/bandpass for baseline wander and powerline interference	Nonlinear phase distorts QRS morphology near the passband edges	Usable for gross baseline removal far from the QRS band, riskier near diagnostic frequency content
IIR with <code>filtfilt</code>	Zero-phase, preserves exact morphology offline	Not deployable in a real-time monitor; doubles effective attenuation	Excellent for offline database curation/training, unsafe as the online algorithm itself
Learned Conv1d	Can adapt to patient-specific or device-specific noise; can be trained end-to-end with the classifier	Requires enough labeled data; no guaranteed phase/magnitude behavior; harder to certify for medical use	Attractive as a research direction, but regulatory/interpretability constraints in medical devices often still favor classical, analyzable filters

B10. Minute-paper synthesis (sample answer)

A zero-phase (`filtfilt`) pipeline uses future samples and effectively squares the filter's magnitude response, so the training data seen by the model is both perfectly time-aligned and more strongly attenuated than anything a causal, single-pass filter can produce in real time. A model evaluated only against this offline-preprocessed data can show excellent validation

accuracy that never has to contend with the causal filter's frequency-dependent delay, weaker attenuation, or start-of-stream edge transients. Once deployed with a causal filter (as it must be, in real time), the input distribution shifts in all of these ways simultaneously, so the reported offline accuracy is an optimistic, misleading estimate of real-world streaming performance.

Part C — Integrative Problem (sample solution sketch)

(a) Recommend a **linear-phase FIR** bandpass/notch for the deployed stage. Two arguments: (1) the fault signature's *onset timing relative to a trigger* is explicitly a feature of interest, so preserving waveform shape/timing via constant group delay (Part A) is more valuable here than the lower order an IIR notch could offer; (2) at 2 kHz sampling with well-separated interference (60 Hz harmonics) and signal band (100–400 Hz), a moderate-length FIR easily achieves the needed selectivity within an acceptable, *known and constant* latency budget, whereas an IIR filter's frequency-dependent group delay (Part B) would introduce an onset-timing error that varies with the exact spectral content of each burst — directly corrupting the feature the classifier relies on.

(b) An unsafe-to-reuse training-only pipeline: notch out the 60 Hz harmonics and bandpass 100–400 Hz using an IIR filter, then apply `scipy.signal.filtfilt` (zero-phase, offline) before computing onset-timing features for every training example. This is unsafe to reuse verbatim at deployment because `filtfilt` requires the complete (including "future") signal and cannot run causally in real time, and because its effective magnitude response is $|H|^2$ of the single-pass filter — a causal deployment filter with the same coefficients would neither match its timing behavior nor its attenuation level.

(c) Validation experiment: take a held-out set of labeled bursts, generate two parallel preprocessed versions — one via the exact offline `filtfilt`/IIR pipeline used in training, one via the actual causal (streaming-compatible) filter intended for deployment — extract the onset-timing feature (and any other relevant features) from both, and compare (i) the classifier's accuracy on each version, and (ii) the distributions of the extracted timing feature itself (e.g., a histogram of the timing offset between the two pipelines' estimated onset for the same underlying event). A statistically significant accuracy drop or a systematic (non-zero-mean) shift in the timing-feature distribution between the two pipelines is direct evidence of train/serve skew that should be fixed (e.g., by switching to a linear-phase FIR or causal-only preprocessing during training, per B8b) before shipping.